

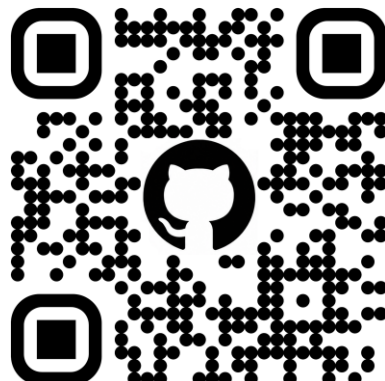
КЕШИ И АБСТРАКЦИИ

Идеальное и реальное кэширование. Переход от С к С++, механизмы объединения и обобщения. Программирование в эпоху нейросетей.

К. Владимиров, 2026
mail-to: konstantin.vladimirov@gmail.com

Как зовут преподавателя?

- Владимиров Константин Игоревич
- Все рабочие вопросы через почту konstantin.vladimirov@gmail.com
- Оперативная связь через telegram [@Tilir](https://t.me/Tilir)
- Слайды на [sourceforge](https://sourceforge.net), примеры кода на [github](https://github.com) (см. QR-коды внизу)



Проблема из реального мира

- Вы пишете программу, которая оперирует миллионами страниц разного размера, различающихся уникальным номером.

```
struct page {  
    int index;    // page index: 1, 2, ... n  
    int sz;       // page size  
    char *data;   // page data  
};
```

- Страницу с номером n вы можете получить по сети медленной функцией.

```
void slow_get_page(int index, struct page *p);
```

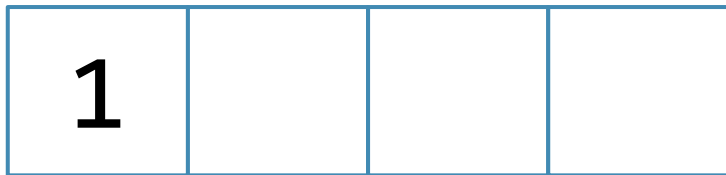
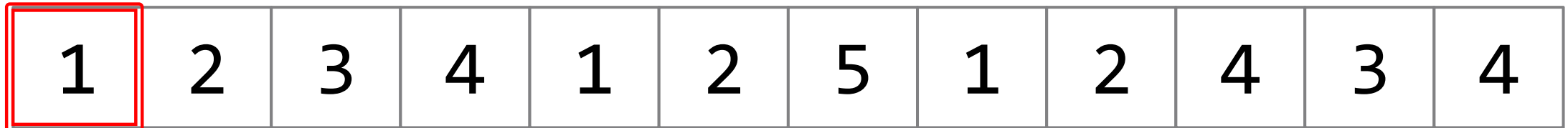
- Но у вас есть место на несколько тысяч страниц локально на диске.

Решение: кеш

- Если у нас есть немного места, его хотелось бы использовать.
- Мы хотим сохранять **некоторые** страницы, раз уж мы не можем сохранить все.
- Небольшая область памяти называется кеш. Туда можно поместить страницу или оттуда можно вытеснить страницу.
- Например у нас есть место на 4 страницы и к нам поступают запросы:
1, 2, 3, 4, 1, 2, 5, 1, 2, 4, 3, 4
- Как будет изменяться кеш? Какую **стратегию** кеширования тут можно выбрать?

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.



Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

2	1		
---	---	--	--

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

3	2	1	
---	---	---	--

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

4	3	2	1
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

1	4	3	2
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

2	1	4	3
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

5	2	1	4
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

1	5	2	4
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

2	1	5	4
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

4	2	1	5
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

3	4	2	1
---	---	---	---

Стратегия LRU (least recently used)

- Если запрошенный элемент найден, он перемещается вперёд.
- Если нет, он помещается вперёд а последний вытесняется.

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

4	3	2	1
---	---	---	---

Обсуждение

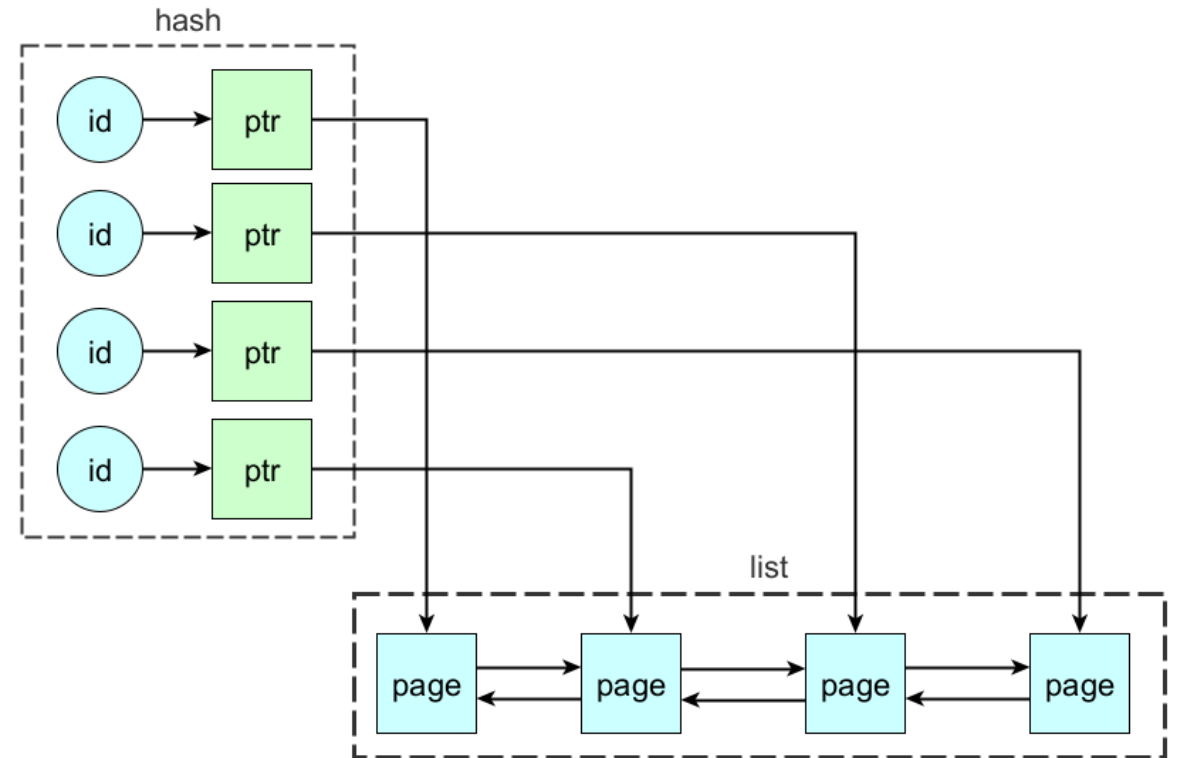
- Какие структуры данных нам понадобятся чтобы реализовать LRU-кеш?
- Давайте попросим нейросеть его сразу и написать?

1	2	3	4	1	2	5	1	2	4	3	4
---	---	---	---	---	---	---	---	---	---	---	---

4	3	2	1
---	---	---	---

Обсуждение

- Какие структуры данных нам понадобятся чтобы сделать LRU cache?
- Двусвязный список для собственно кеша.
- Хеш-таблица для того, чтобы быстро определять кеширован ли элемент.



Первый нейровариант: всё в main

- Промпт вполне естественный.

Сделай в этой папке референсную реализацию LRU на языке C.

На `stdin` приходит размер кеша и кол-во элементов, потом сами элементы (целыми числами). На `stdout` количество попаданий.

Пример. Вход: 2 6 1 2 1 2 1 2, выход: 4

Не забудь сборку и тесты.

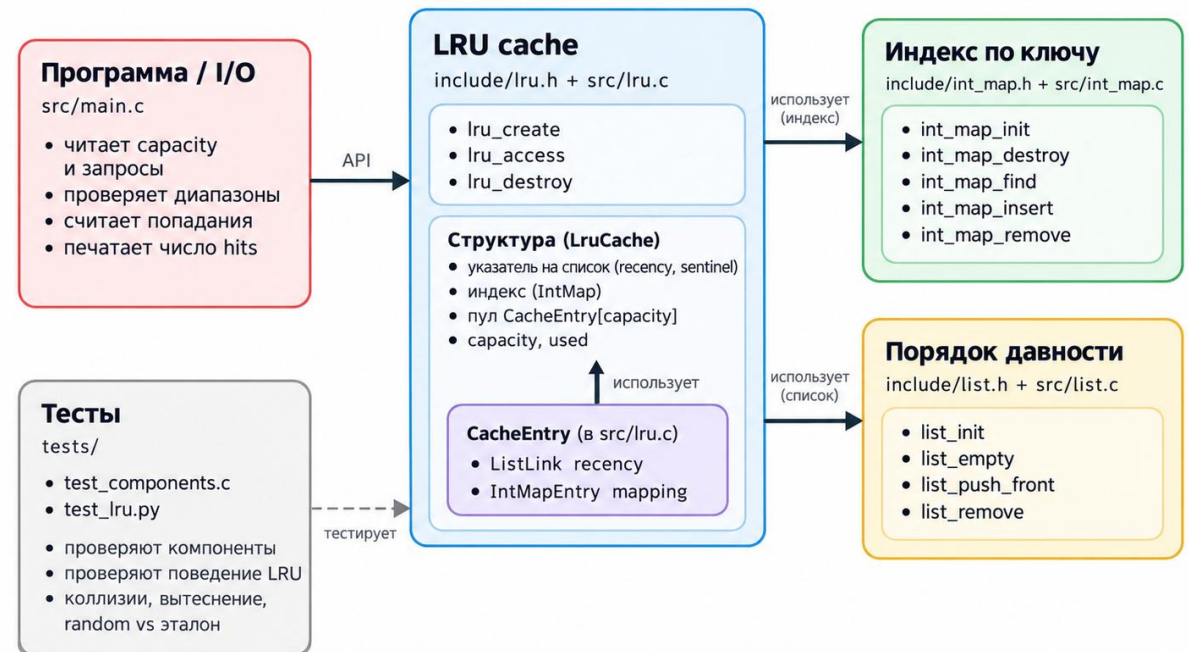
- Модель свежайшая: chatgpt-6-astra в агентском режиме через codex, уровень размышления максимальный, токенов приказано не жалеть.
- Выхлоп: довольно печальный (см. по ссылке).

Подумаем о переиспользовании

- Весь код слитый в один файл main.c это очень и очень грустная идея.
- Допустим нам кто-то рассказал о структурах данных. Мы можем попросить:

Сделай лучшее решение с разбиением на файлы и выделением структур данных в отдельные переиспользуемые компоненты.

- Она вам даже нарисует (ниже бред, но красиво).



Видим ли мы тут проблемы?

```
bool lru_access(LruCache *cache, int64_t key) {
    if (!cache->capacity) return false;
    IntMapEntry *found = int_map_find(&cache->index, key);
    CacheEntry *entry;
    if (found) {
        entry = found->value; list_remove(&entry->recency);
    } else {
        if (cache->used < cache->capacity) {
            entry = &cache->entries[cache->used++];
            list_init(&entry->recency);
        } else {
            entry = (CacheEntry *)cache->recency.prev;
            list_remove(&entry->recency); int_map_remove(&cache->index, &entry->mapping);
        }
        entry->mapping.key = key; entry->mapping.value = entry;
        bool inserted = int_map_insert(&cache->index, &entry->mapping); assert(inserted);
    }
    list_push_front(&cache->recency, &entry->recency);
    return found != NULL;
}
```

Да проблемы явно есть

- Размывание уровней абстракции.

```
entry = &cache->entries[cache->used++];  
list_init(&entry->recency);
```

- Проверка ошибок ассертами.

```
inserted = int_map_insert(&cache->index, &entry->mapping);  
assert(inserted);
```

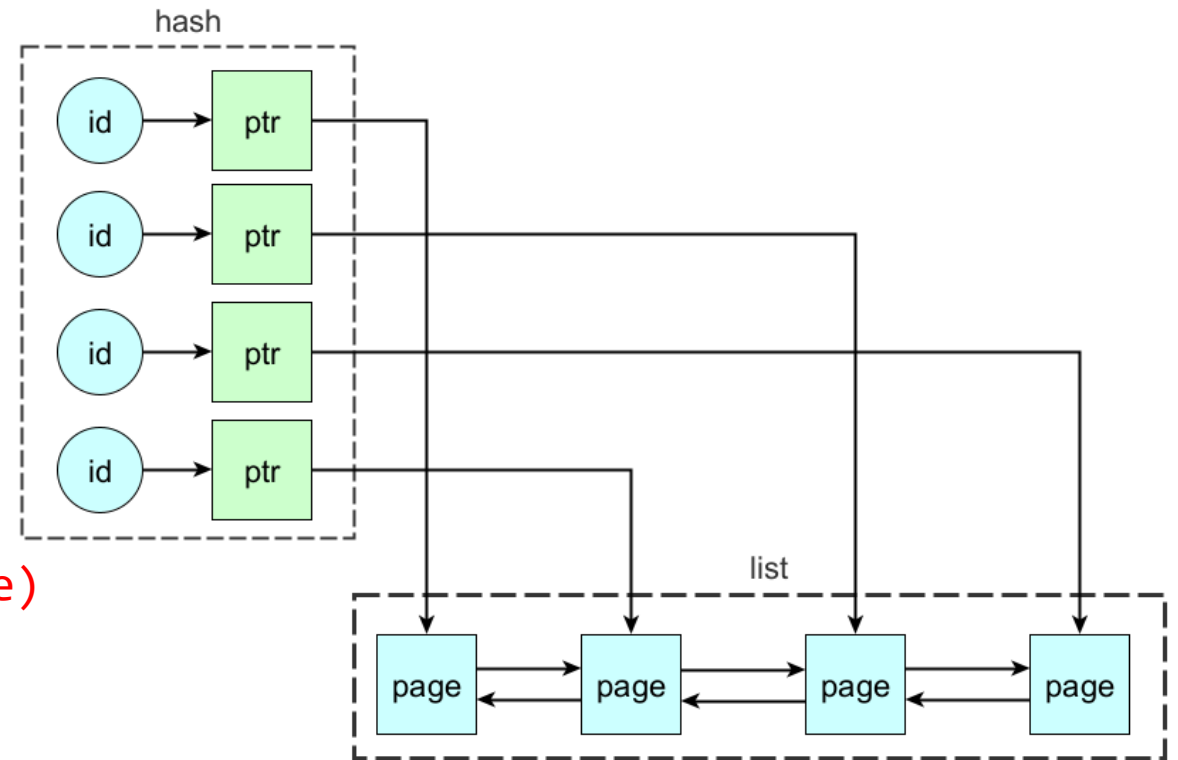
- И увы, эта абстракция всё ещё бесполезна для переиспользования.

```
bool lru_access(LruCache *cache, int64_t key) { ... }
```

- Зачем нам вообще кеш если мы уже знаем ключ?

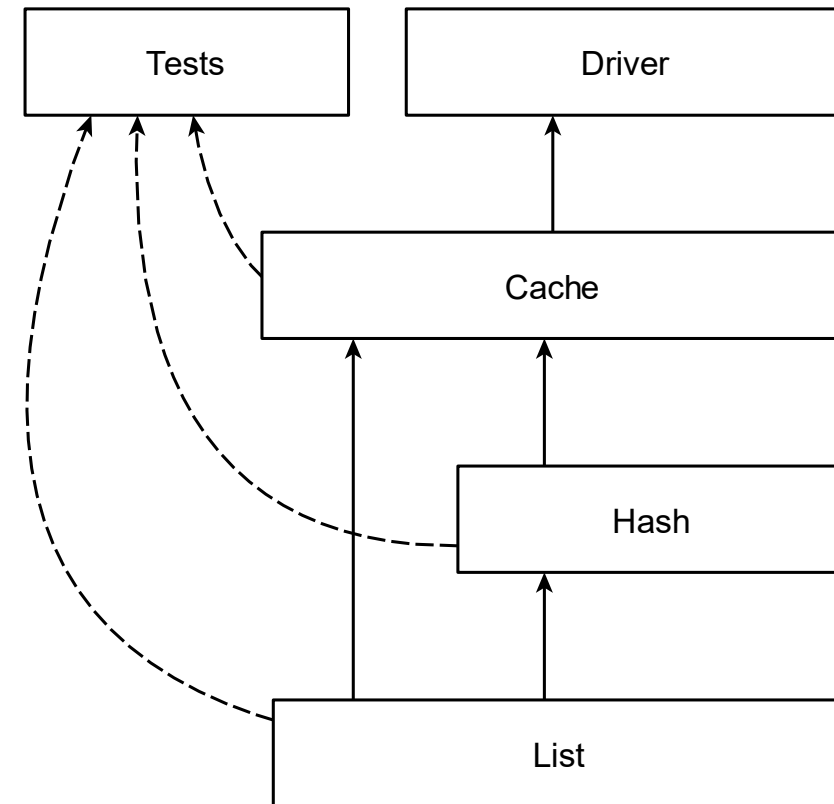
Алгоритм LRU – псевдокод

```
next_page = get_page_number()
while next_page != NO_PAGE do
  page_node = is_cached(next_page)
  if page_node != nil
    list_move(page_node, list_top)
  else
    if cache_is_full()
      list_delete(list_bottom)
    end if
    page_node = slow_get_page(next_page)
    list_add(page_node, list_top)
  end if
  process_page(page_node)
  next_page = get_page_number()
end while
```



Проектирование

- Перед тем как написать программу, мы её проектируем, разделяя на логические слои.
- Каждый слой работает с открытым интерфейсом другого слоя, но не с деталями его реализации.
- Это позволяет снизить интеллектуальную нагрузку.
- Существенная часть программирования это **борьба со сложностью**.



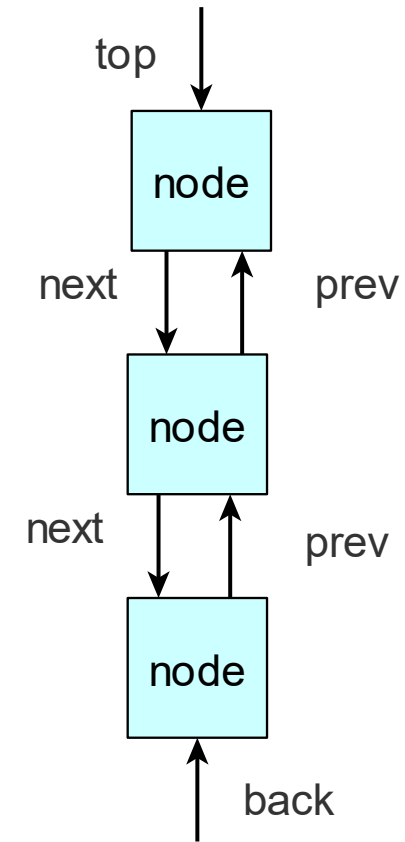
LRU на языке C: двусвязный список

// скрытая реализация

```
struct list_node_t {  
    struct list_node_t *next;  
    struct list_node_t *prev;  
    struct page_t *data;  
};
```

```
struct list_t {  
    struct list_node_t *top;  
    struct list_node_t *back;  
};
```

- Какие интерфейсные функции нам могут понадобиться?



LRU на языке C: двусвязный список

// интерфейс в заголовочном файле

```
struct list_t;
```

```
struct list_t *list_create();
```

```
int list_size(const struct list_t *lst);
```

```
struct page_t *list_back(const struct list_t *lst);
```

```
void list_pop_back(struct list_t *lst);
```

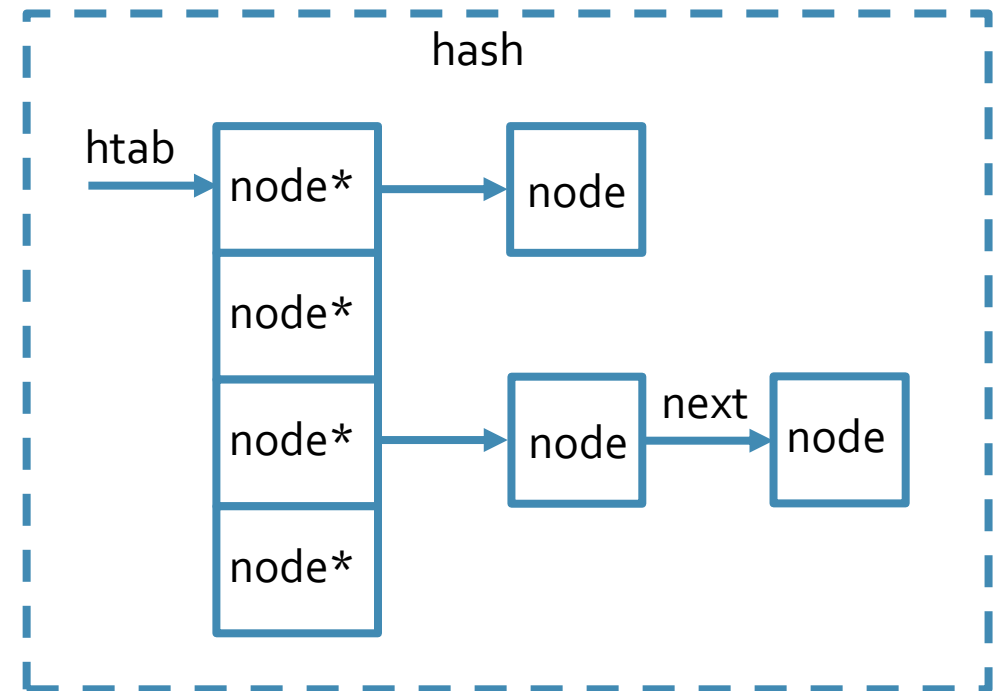
```
void list_push_front(struct list_t *lst, struct page_t *q);
```

```
void list_move_upfront(struct list_t *lst, struct list_node_t *p);
```

```
void list_free(struct list_t *lst);
```

LRU на языке C: хеш-таблица

```
struct hashmap_entry_t {  
    int key;  
    list_node_t *node;  
};  
  
struct hashmap_node_t {  
    struct hashmap_node_t *next;  
    struct hashmap_entry_t entry;  
};  
  
struct hash_t {  
    struct hashmap_node_t **htab;  
    int len;  
};
```



LRU на языке C: хеш-таблица

// создаёт пустой хеш с заданным количеством бакетов

`struct hash_t *htable_create(int len);`

// ищет узел по ключу

`struct list_node_t *htable_find(struct hash_t *h, int key);`

// вставляет пару ключ+значение в хеш

`void htable_insert(struct hash_t *h, int key, struct page_t *data);`

// стирает пару ключ+значение из хеша по ключу

`void htable_erase(struct hash_t *h, int key);`

// освобождает хеш и всю использованную память

`void htable_free(struct hash_t *h);`

LRU на языке C: собственно кеш

```
struct cache_t *cache_create(int cache_size);
void cache_free(struct cache_t *c);

typedef struct page_t * (*slow_get_page_t)();

// поискать или вставить страницу возвращает true если hit
bool cache_lookup_update(
    struct cache_t *c, int key, slow_get_page_t slow);

// скрытая реализация
struct cache_t {
    int sz;
    struct hash_t *hash; struct list_t *lst;
};
```

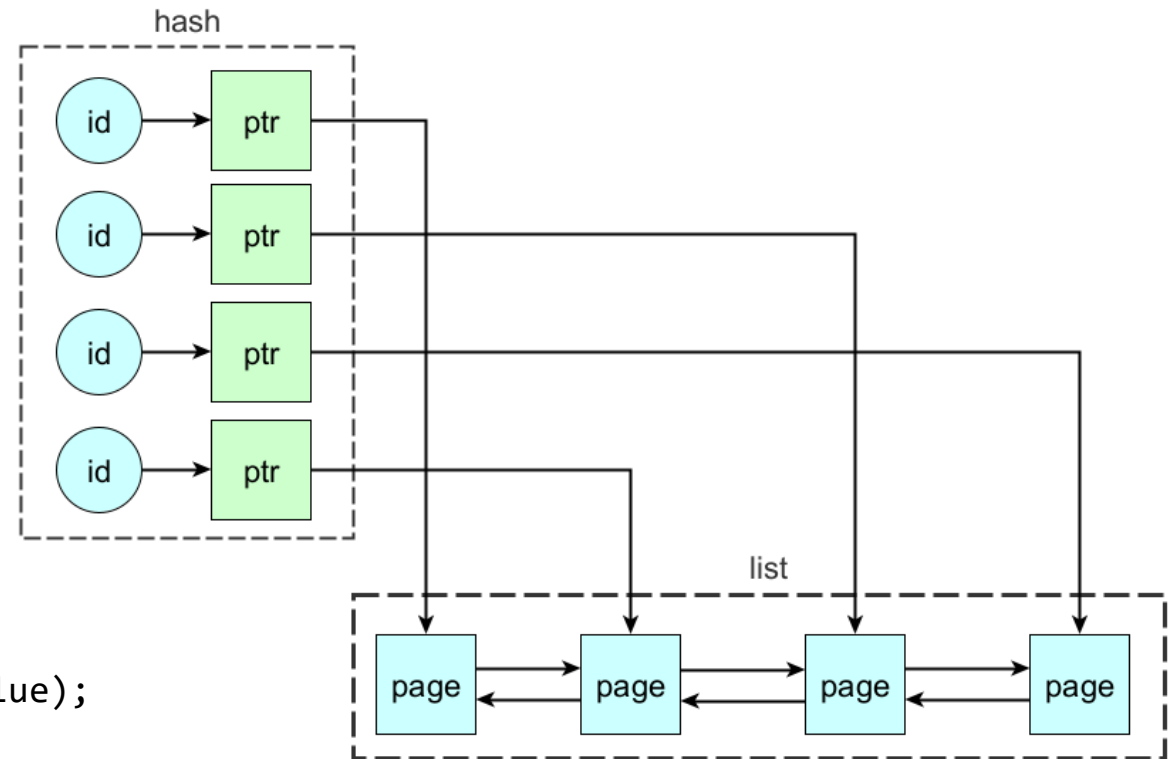
Алгоритм LRU – реализация

```
LruResult lookup_update(LruCache *cache, PageId key,
                       SlowGetPage f, Page *value) {
    HashMapValue found;
    if (!cache || !value || !f)
        return LRU_INVALID_ARGUMENT;

    if (!cache->capacity) {
        *value = f(key);
        return LRU_MISS;
    }

    if (hash_map_find(cache->index, key, &found)) {
        ListNode *node = found;
        list_move_front(cache->recency, node);
        const CacheEntry *entry = list_value(node);
        *value = entry->value;
        return LRU_HIT;
    }

    return insert_page(cache, key, slow_get_page, value);
}
```



Владение этим кодом тоже не сахар

- Хороша ли получившаяся программа?

```
int64_t hits = 0;
for (int64_t i = 0; i < count; ++i) {
    PageId key; Page page;
    if (!read_integer(&key)) {
        fprintf(stderr, "Expected int at %" PRIu64 ".\n", i + 1);
        lru_destroy(cache); return EXIT_FAILURE;
    }
    LruResult result = lookup_update(cache, key, &slow_get_page, &page);
    if (result != LRU_HIT && result != LRU_MISS) {
        fprintf(stderr, "Cannot fetch page (error %d).\n", (int)result);
        lru_destroy(cache); return EXIT_FAILURE;
    }
    hits += result == LRU_HIT;
}
```

Обсуждение

- Хороша ли получившаяся программа?
- Нет, она довольно плоха:
 - Она слишком сложна и вы теперь в заложниках у вашей нейросети.
 - В ней есть тяжёлые и неприятные части (например хеш-таблица довольно наивная и скорее всего не потянет серьёзные испытания на прочность).
 - В ней приходится постоянно следить за памятью, например простой способ организовать утечку это забыть вызвать одну функцию.
- Ну и наконец... а что если нам также надо кешировать **другой тип страниц?**

Неверный выбор инструмента

- Язык С является мощным и совершенным инструментом.
- Но по своей природе он **слишком низкоуровневый**.
- Он исключительно хорош, чтобы писать код в пределах нескольких функций.
- Но это плохой выбор, для надёжных переиспользуемых абстракций.
 - Иногда крупные системы, построенные на С, работают, например Linux kernel или PostgreSQL. Но это получается как раз из-за максимально конкретной предметной области.
- Он для этого непригоден до такой степени, что у него даже нет нормальных структур данных в стандартной библиотеке.

Генерация не всегда спасает

- Да нейросеть способна сгенерировать нам «хеш для конкретного типа».
- Но она не способна (если ограничить её языком C) выразить концепцию «хеш для любого типа».
 - Поэтому она будет раздувать ваш проект, вылезать из контекста и тратить токены.
- Владение сгенерированным кодом это всё ещё ваша ответственность.
 - В мире LLM код стал дешёвым.
 - Состояния программы, инварианты, интерфейсы, ownership, корректность, возможность изменения и цена проверки дешёвыми не стали.

C++: inception

- *Bjarne Stroustrup* разработал C++ в 1983 году во время работы в AT&T Bell Labs для облегчения работы над крупномасштабными задачами.
- Изначально C++ компилировался в язык C и первый компилятор назывался Cfront.
- До сих пор общее подмножество C и C++ это почти весь C.
- Прежде, чем написать LRU cache на C++ нам потребуется ввести несколько новых концепций.

Объединение данных и методов

- Одной из главных особенностей C++ является объединение данных и методов их обработки.

```
struct Point { double x, y; };
```

```
struct Triangle { struct Point pts[3]; }; // C way
```

```
double square(const struct Triangle *pt) {  
    double sq = pt->pts[0].x * (pt->pts[1].y - pt->pts[2].y) +  
                pt->pts[1].x * (pt->pts[2].y - pt->pts[0].y) +  
                pt->pts[2].x * (pt->pts[0].y - pt->pts[1].y);  
    return abs(sq) / 2.0;  
}
```

Объединение данных и методов

- Одной из главных особенностей C++ является объединение данных и методов их обработки.

```
struct Point { double x, y; };

struct Triangle {
    Point pts[3];    // struct not required before Point

    double square() {
        double sq = pts[0].x * (pts[1].y - pts[2].y) +
                     pts[1].x * (pts[2].y - pts[0].y) +
                     pts[2].x * (pts[0].y - pts[1].y);
        return abs(sq) / 2.0;
    }
};
```

Объединение данных и методов

- Не обязательно определять функцию внутри.

```
struct Triangle {  
    Point pts[3];  
    double square() const; // const так как не модифицирует поля  
};  
  
double Triangle::square() const {  
    double sq = pts[0].x * (pts[1].y - pts[2].y) +  
                pts[1].x * (pts[2].y - pts[0].y) +  
                pts[2].x * (pts[0].y - pts[1].y);  
    return abs(sq) / 2.0;  
}
```

Объединение данных и методов

- Использование получившегося кода.

```
Triangle t; // объект типа Triangle
```

```
t.pts[0] = Point {1.0, 1.0}; // запись поля
```

```
t.pts[1] = Point {3.0, 3.0};
```

```
t.pts[2] = Point {1.0, 2.0};
```

```
double a = t.square(); // вызов метода
```

- Таким образом, с методами занесёнными внутрь структуры, мы работаем так же как и с полями через точку либо через стрелочку.

Указатель на объект

- Хорошо спроектированная структура данных на С часто также берёт "указатель на себя" первым параметром.
- Делая его неявным, мы как бы говорим "сделай для себя".

```
double square(const struct Triangle *pt) {  
    double sq = pt->pts[0].x * (pt->pts[1].y - pt->pts[2].y) +  
                pt->pts[1].x * (pt->pts[2].y - pt->pts[0].y) +  
                pt->pts[2].x * (pt->pts[0].y - pt->pts[1].y);  
    return abs(sq) / 2.0;  
}
```

```
Triangle t; t.square(); // this == &t
```

Использование неявного this

- Хорошо спроектированная структура данных на С часто также берёт "указатель на себя" первым параметром.
- Делая его неявным, мы как бы говорим "сделай для себя".

```
double Triangle::square() const {  
    double sq = this->pts[0].x * (this->pts[1].y - this->pts[2].y) +  
               this->pts[1].x * (this->pts[2].y - this->pts[0].y) +  
               this->pts[2].x * (this->pts[0].y - this->pts[1].y);  
    return abs(sq) / 2.0;  
}
```

- Указывать явный this иногда необходимо. Но делать это без необходимости -- это дурной тон.

Использование неявного this

- Хорошо спроектированная структура данных на С часто также берёт "указатель на себя" первым параметром.
- Делая его неявным, мы как бы говорим "сделай для себя".

```
double Triangle::square() const {  
    double sq = pts[0].x * (pts[1].y - pts[2].y) +  
                pts[1].x * (pts[2].y - pts[0].y) +  
                pts[2].x * (pts[0].y - pts[1].y);  
    return abs(sq) / 2.0;  
}
```

- Указывать явный this иногда необходимо. Но делать это без необходимости -- это дурной тон.

Обсуждение

- Есть ли разница между C-style и C++-style square?

```
double Triangle::square() const {  
    // ....  
}  
  
double square(const struct Triangle *pt) {  
    // ....  
}
```

Обобщение данных и методов

- Ещё одна важная концепция это обобщение через механизм шаблонов.
- Конкретный тип: точка из двух целых координат.

```
struct Point_int { int x, y; };
```

```
Point_int p;
```

- Обобщённый тип: точка из двух **любых** координат.

```
template <typename T> struct Point { T x, y; };
```

```
Point<int> pi;
```

```
Point<double> pd;
```

Обобщение данных и методов

- Тот же треугольник можно обобщить на любые типы точек.

```
template <typename T> struct Point { T x, y; };  
  
template <typename U> struct Triangle {  
    Point<U> pts[3]; // U будет подставлено как T в Point  
  
    // почему я изменил интерфейс на double_square?  
    U double_square() const {  
        U sq = pts[0].x * (pts[1].y - pts[2].y) +  
                pts[1].x * (pts[2].y - pts[0].y) +  
                pts[2].x * (pts[0].y - pts[1].y);  
        return (sq > 0) ? sq : -sq; // почему больше не abs?  
    }  
};
```

Обсуждение

- С одной стороны обобщение создаёт возможности.

```
Triangle<double> t;  
Triangle<float> tf;
```

- С другой стороны оно создаёт проблемы.
- Это очень часто ходит рука об руку.

Обобщение функций

- Пожалуй единственным способом написать на С максимум двух чисел является макрос.

```
#define MAX(x, y) (((x) > (y)) ? (x) : (y))
```

- Перечислите все проблемы в этом макросе.

Обобщение функций

- Пожалуй единственным способом написать на С максимум двух чисел является макрос.

```
#define MAX(x, y) (((x) > (y)) ? (x) : (y))
```

- Перечислите все проблемы в этом макросе.
- На С++ шаблон функции лишён этих проблем.

```
template <typename T> T max(T x, T y) {  
    return (x > y) ? x : y;  
}
```

Обобщение вместо void*

- Стандартная функция из библиотеки C.

```
void qsort(void* base, size_t num, size_t size,  
          int (*compar)(const void*, const void*));
```

- Что можно с ней сделать используя шаблоны?

Обобщение вместо void*

- Первая итерация.

```
template <typename T, typename Comp>  
void qsortpp(T* base, size_t num, Comp compare);
```

- Вместо передачи указателя и длины, можно передавать два указателя на начало и конец интервала.

Обобщение вместо void*

- Вторая итерация.

```
template <typename T, typename Comp>  
void qsortpp(T* start, T* fin, Comp compare);
```

- Вместо указателей можно использовать указателе-подобные объекты, так называемые итераторы и получить.

```
template <typename It, typename Comp = std::less<>>  
void sort(It start, It fin, Comp compare);
```

Обсуждение

- Что будет работать быстрее?

```
qsort(narr, nelts, sizeof(int), intless);
```

или

```
std::sort(narr, narr + nelts);
```

- Давайте поставим эксперимент и попробуем истраковать результат.

Бенчмаркинг

- Бенчмаркингом называется тщательная подготовка небольших программ для измерения производительности.

// подготовка

```
int *narr = (int *) calloc(nelts, sizeof(int));
```

// время измеряется только для этого цикла

```
for (auto _ : state) {  
    for (int i = 0; i < nelts; ++i)  
        narr[i] = rand() % nelts;  
    qsort(narr, nelts, sizeof(int), intless);  
}
```

- Покритикуйте (и позащищайте) этот бенчмарк.

Стандартная библиотека

- С++ имеет массу стандартных обобщённых **контейнеров** и обобщённых **алгоритмов** над ними.
- Например `std::list` это стандартный двусвязный список.
- Работа с ним не сложнее, чем с самописным треугольником.

```
Triangle<double> t; // создать треугольник
```

```
t.pts = {{1, 0}, {2, 1}, {3, 2}}; // задать точки
```

```
double sq = t.double_square() / 2; // вычислить площадь
```

Стандартная библиотека

- C++ имеет массу стандартных обобщённых **контейнеров** и обобщённых **алгоритмов** над ними.
- Например `std::list` это стандартный двусвязный список.
- Работа с ним не сложнее, чем с самописным треугольником.

```
std::list<int> lst; // создать список
```

```
lst.push_back(2); // добавить несколько узлов
```

```
lst.push_back(1);
```

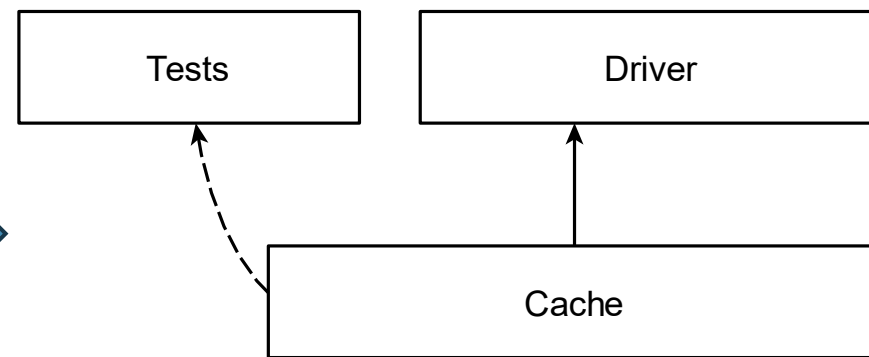
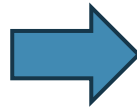
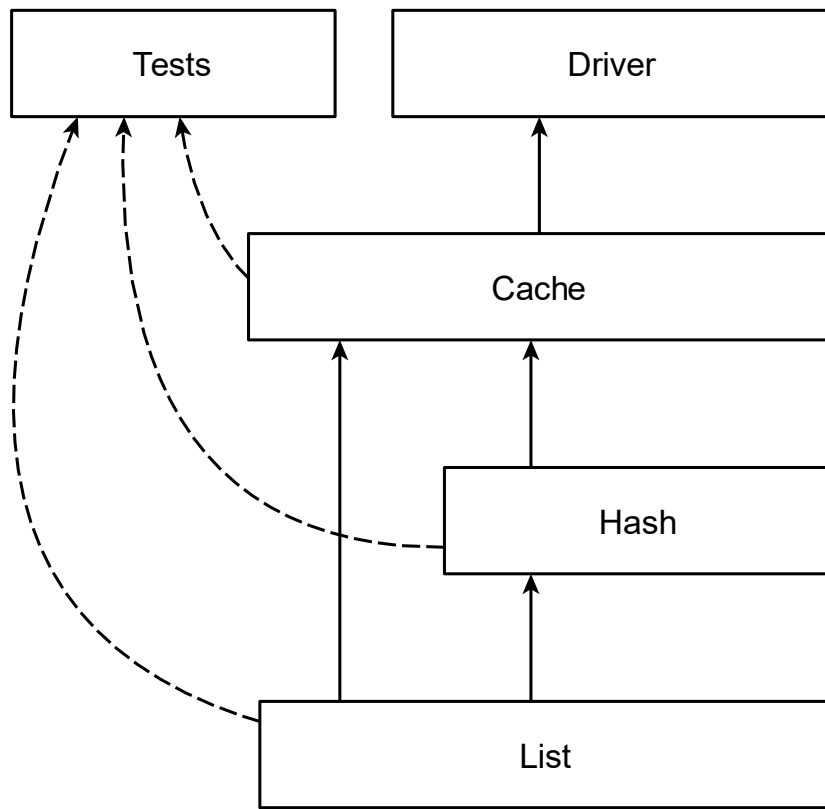
```
lst.push_front(1); // {1, 2, 1}
```

```
lst.remove_if(1); // удалить все единицы
```

LRU на языке C++

- Всё несколько упрощается.
 - Можно взять `list` в качестве двусвязного списка.
 - Можно взять `unordered_map` в качестве хеш-таблицы.
 - Написать уровень абстракции LRU кеша.
 - Написать к нему тесты.
- Это один модуль (и один заголовочный файл).
- И это уже сразу обобщённый переиспользуемый код.

Проектирование



- Развитая стандартная библиотека упрощает проектирование.
- Она возможна только за счёт лучших механизмов абстракции.

Скетч: LRU на языке C++

```
template <typename T, typename KeyT = int>
struct cache_t {
    size_t sz_;
    std::list<T> cache_;

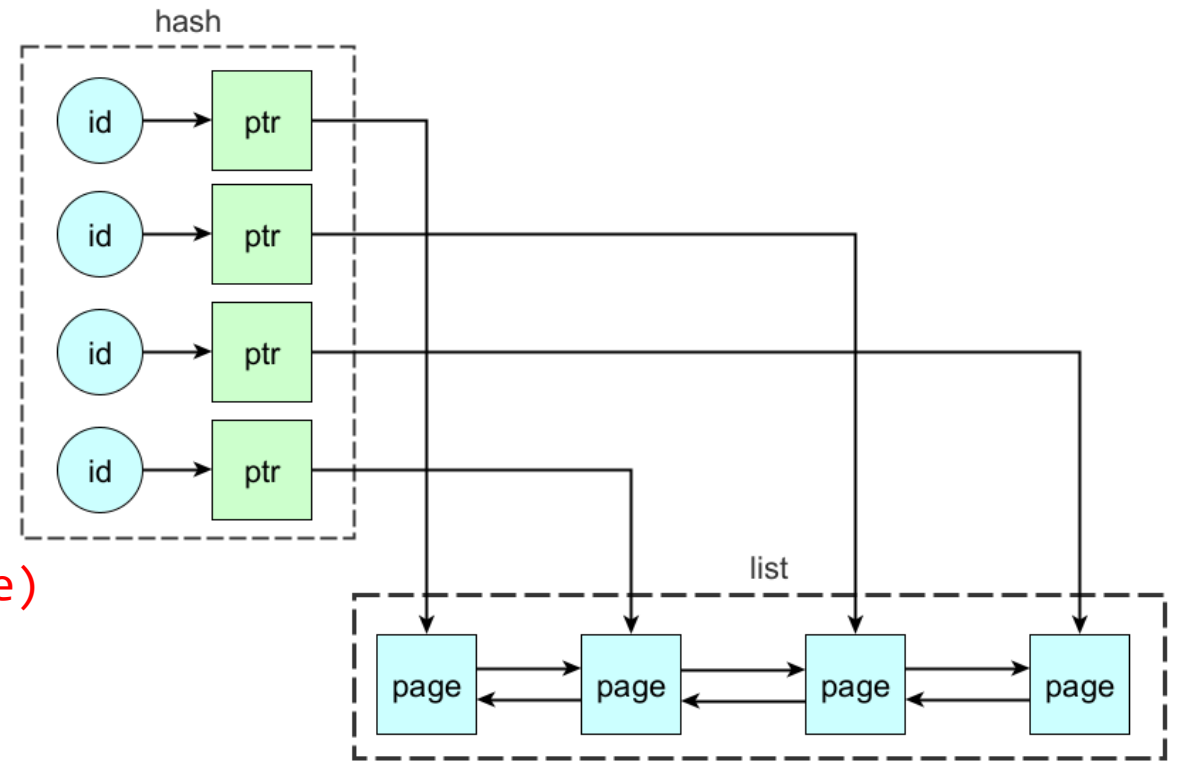
    using ListIt = typename std::list<T>::iterator;
    std::unordered_map<KeyT, ListIt> hash_;

    bool full() const;

    template <typename F>
    bool lookup_update(KeyT key, F slow_get_page);
};
```

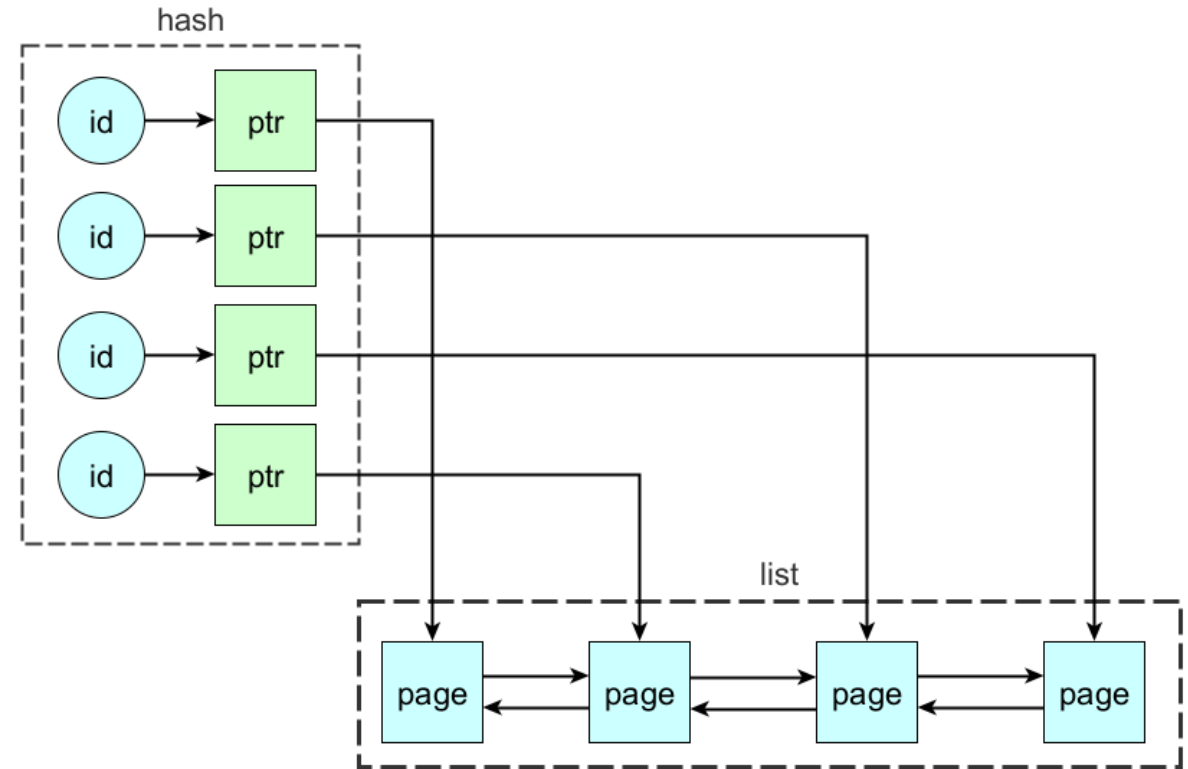
Алгоритм LRU – псевдокод

```
next_page = get_page_number()
while next_page != NO_PAGE do
  page_node = is_cached(next_page)
  if page_node != nil
    list_move(page_node, list_top)
  else
    if cache_is_full()
      list_delete(list_bottom)
    end if
    page_node = slow_get_page(next_page)
    list_add(page_node, list_top)
  end if
  process_page(page_node)
  next_page = get_page_number()
end while
```



Алгоритм LRU – реализация

```
template <typename F> bool lookup_update(KeyT key, F slow_get_page) {  
    if (max_capacity() == 0)  
        return false;  
  
    auto hit = hash_.find(key);  
    if (hit != hash_.end()) {  
        auto eltit = hit->second;  
        cache_.splice(cache_.begin(), cache_, eltit);  
        return true;  
    }  
  
    T page = slow_get_page(key);  
    if (full()) {  
        hash_.erase(cache_.back().first);  
        cache_.pop_back();  
    }  
  
    cache_.emplace_front(key, page);  
    hash_.emplace(key, cache_.begin());  
    return false;  
}
```



Как быть с присвоением размера?

```
cache_t<page_t> c; // создали кеш
std::cin >> m >> n;
c.sz_ = m; // только тут присвоили размер
for (int i = 0; i < n; ++i) {
    page_t p;
    std::cin >> p.id;
    assert(std::cin.good());
    if (c.lookup_update(&p, slow_get_page)) hits += 1;
}
std::cout << hits << "\n";
```

Обсуждение

- Получившаяся программа всё ещё плоха с точки зрения C++.
 - Структура кеша не **инкапсулирует** контекст и не создаёт законченной абстракции.
 - Мы не написали **конструктор**, а вместо этого присваиваем `sz` напрямую.
- Но она уже гораздо лучше: короче, яснее, надёжней и вы можете использовать этот кеш чтобы кешировать в принципе любые данные.
- К тому же в неё легко вносить изменения и оптимизации.
 - Скажем добавить контроль размера кеша как поле в класс а не вычислять каждый раз размер списка.

Идея конструктора

- Этот участок кода очень подозрительный.

```
cache_t<page_t> c;  
std::cin >> m >> n; // scanf("%d%d", &m, &n);  
c.SZ_ = m;
```

- Хочется сделать параметр размера обязательным при создании кеша.

Идея конструктора

- Напишем простой конструктор.

```
template <typename T, typename KeyT = int>
struct cache_t {
    size_t sz_;
    // .....
    cache_t(size_t sz) : sz_(sz) {} // ctor
```

- Теперь попытка создать кеш без параметра -- это ошибка.

```
cache_t<page_t> c; // FAIL
```

Идея конструктора

- Напишем простой конструктор.

```
template <typename T, typename KeyT = int>
struct cache_t {
    size_t sz_;
    // .....
    cache_t(size_t sz) : sz_(sz) {} // ctor
```

- Мы создаём кеш **после** того как у нас есть вся информация.

```
std::cin >> m >> n;
cache_t<page_t> c{m}; // creating with ctor
```

Value-семантика

- Хорошо написанный тип обладает тем, что называется **семантикой значения**.
- Грубо говоря, он "ведёт себя как int".
 - Его контекст скрыт внутри, изменение других объектов его не затрагивает
 - Константные операции над ним делают всё, что можно сделать.
- Конструкторы это первый шаг к тому, чтобы сделать ваши типы (такие как "кеш" или "треугольник") такими же естественными как int и float.

Мы только потрогали воду лапкой

- Язык C++ богат возможностями.
- Если вы просто знаете C, то многим вы можете начинать пользоваться прямо сейчас.
- Но увы, язык C++ сложен и коварен.
- Использовать его, не обладая глубоким пониманием происходящего часто бывает неприятно.



C++ за 21 день?

- Надо осознавать, что мы находимся в начале **долгого** пути и настраиваться на марафонскую дистанцию.
- Я надеюсь, на втором курсе мы успеем разобрать:
 - Базовое объектно-ориентированное программирование.
 - Простые шаблоны классов и функций.
 - Основные механизмы стандартной библиотеки.
 - Много интересных алгоритмов и структур данных.
- Последний пункт особенно важен. Наверное можно механически выучить язык не научившись программированию на этом языке. Но это бессмысленно.

Days 1 - 10

Teach yourself variables, constants, arrays, strings, expressions, statements, functions,...



Days 11 - 21

Teach yourself program flow, pointers, references, classes, objects, inheritance, polymorphism,



Days 22 - 697

Do a lot of recreational programming. Have fun hacking but remember to learn from your mistakes.



Days 698 - 3648

Interact with other programmers. Work on programming projects together. Learn from them.



Days 3649 - 7781

Teach yourself advanced theoretical physics and formulate a consistent theory of quantum gravity.



Days 7782 - 14611

Teach yourself biochemistry, molecular biology, genetics,...



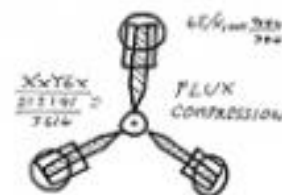
Day 14611

Use knowledge of biology to make an age-reversing potion.



Day 14611

Use knowledge of physics to build flux capacitor and go back in time to day 21.



Day 21

Replace younger self.



О домашних работах

- Начиная с этого года они стали существенно сложнее, так как нейросети существенно расширяют ваши возможности.
- Проекты всё ещё можно делать в командах.
 - Не забывайте в почте ставить в СС (carbon copy) членов команды.
- Система приёма проектов **коллоквиальная**.
 - Вам назначается куратор и вы (когда готовы) согласуете с ним время сдачи.
 - Вы защищаете проект очно и без возможности спросить о чём-то нейросеть.
 - Проверяется в том числе ваше знание собственного кода.
- У домашних работ есть скрытые уровни и неожиданное развитие.
 - Подробности после успешной защиты.

Как преуспеть в домашних работах

- Организация кода.
 - Убедитесь, что вы умеете работать с системой контроля версий, что ваши программы разбиты на переиспользуемые модули, что модули логично названы.
- Система сборки.
 - Лучше всего если вы освоите make, но любая другая система тоже подойдёт. Проверьте сборку под Windows и под Linux.
- Тестирование и обработка ошибок ввода.
 - Не бойтесь писать и даже генерировать end-to-end тесты: вход, выход.
- Качество кода.
 - Убедитесь, что ваши методы достаточно короткие, что их имена логичны, что на не модифицирующих методах не забыт const.

Идеальное кэширование

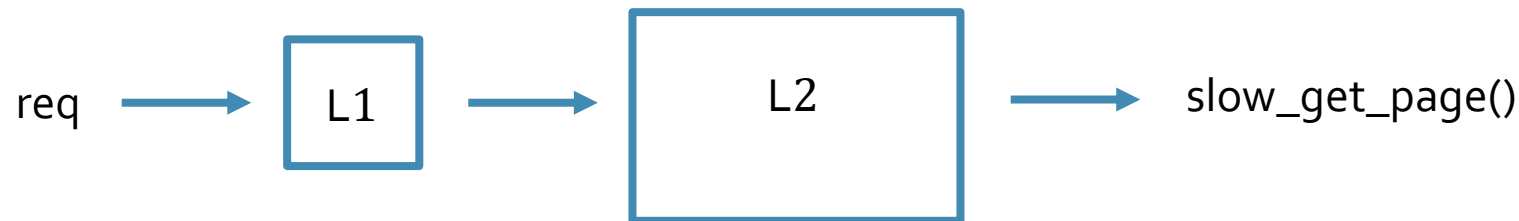
- Представим, что мы заранее знаем будущее.
- Тогда оптимальная стратегия состоит в следующем:
 - для каждой следующей страницы посмотреть в будущем когда она встретится
 - выкинуть из кеша ту, которая встретится **позже прочих**.

1	2	3	4	1	2	5	1	3	5	5	4
---	---	---	---	---	---	---	---	---	---	---	---

1	3	4	5
---	---	---	---

Многоуровневое кеширование

- У вас может быть несколько уровней более быстрого хранилища.
- Например: что-то держать в оперативной памяти (L1) а что-то на диске (L2).
- В таких случаях система может иметь разные алгоритмы вытеснения на разных уровнях.



Домашняя работа NHWC – кеши

- Исследуйте алгоритмы кеширования из перечисленных ниже
 - **ARC** (adaptive replacement cache)
 - **2Q** (алгоритм двух очередей, дальнейшее развитие LRU)
 - **LFU** (вытеснение наименее часто используемого)
 - **LIRS** (low-inference recency set)
- Постройте многоуровневую систему и сравните её с идеальным кешированием.
- Сгенерируйте интересные паттерны и попробуйте найти оптимальную систему.
- Не забудьте о тестах.

Формальные требования

- Конфиг (чтение из файла). Количество уровней и схема каждого уровня.

1 LRU

2 LFU ARC

- Входные данные: размер кеша, размер данных и далее данные.

2 4 1 2 1 2

2 10 1 2 3 1 2 4 5 6 4 5

- В обычном режиме на выход только число хитов.

Литература

- [CXX20] ISO/IEC 14882 – "Information technology – Programming languages – C++", 2020
- [SB] Bjarne Stroustrup, The C++ Programming Language (4th Edition), 2013
- [C11] ISO/IEC 9899 – "Information technology – Programming languages – C", 2011
- [K&R] Brian W. Kernighan, Dennis Ritchie – The C programming language, 1988
- [LS] Stanley Lippman, The C++ Primer (5th Edition), 2012

